SpiceVeg Agri — Complete Build Prompt for AI Agent

Static GitHub Pages + Firebase Firestore | QR Seed Label System

---

PROJECT OVERVIEW

Build a **single-file static web app** (`index.html`) for **SpiceVeg Vegetable Seeds** to be hosted on GitHub Pages at: `https://krishnakoushik9.github.io/Spice-Veg-Agri-Customer/`

The app has two modes:

1. **Admin Mode** — Password-protected. Enter seed batch details, generate a QR code, save to Firebase Firestore, view all past entries.
2. **Customer Mode** — Opened via QR scan. Displays read-only seed packet details for that specific QR. Shows cultivation techniques on button press.

All in one `index.html` **file.** No build tools. No npm. No React. Pure HTML + CSS + Vanilla JS. Must load fast on 2G/3G Indian rural networks.

TECH STACK

- **HTML/CSS/JS** — single file, no frameworks
- **Firebase Firestore** — via direct REST API (`https://firestore.googleapis.com/v1/...`) — NO Firebase SDK
- **QRCode.js** — via CDN: `https://cdnjs.cloudflare.com/ajax/libs/qrcodejs/1.0.0/qrcode.min.js`
- **Font** — Google Fonts: `DM Sans` (body), `Literata` (brand name) — load asynchronously

---

FIREBASE CONFIG

```javascript
const FB = {
  apiKey: "AIzaSyCXx0h_4FVtBnM83-QRP4MhwPB3juiDSr4",
  projectId: "spice-veg-agri"
};
const FS_BASE = `https://firestore.googleapis.com/v1/projects/${FB.projectId}/data
```

Firestore REST helpers (paste-ready)

```javascript
async function fsSet(collection, docId, data) {
  const fields = {};
  for (const [k, v] of Object.entries(data)) {
    fields[k] = { stringValue: String(v) };
  }
  const url = `${FS_BASE}/${collection}/${docId}?key=${FB.apiKey}`;
  const res = await fetch(url, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ fields })
  });
  return res.json();
}

async function fsGet(collection, docId) {
  const url = `${FS_BASE}/${collection}/${docId}?key=${FB.apiKey}`;
  const res = await fetch(url);
  if (!res.ok) return null;
  const doc = await res.json();
  const out = {};
  if (doc.fields) {
    for (const [k, v] of Object.entries(doc.fields)) {
      out[k] = v.stringValue ?? v.integerValue ?? v.booleanValue ?? '';
    }
  }
  return out;
}

async function fsList(collection) {
  const url = `${FS_BASE}/${collection}?key=${FB.apiKey}`;
  const res = await fetch(url);
  const json = await res.json();
  if (!json.documents) return [];
  return json.documents.map(doc => {
    const out = { _id: doc.name.split('/').pop() };
    for (const [k, v] of Object.entries(doc.fields || {})) {
      out[k] = v.stringValue ?? v.integerValue ?? '';
    }
    return out;
  });
}
```

Collection name: `seed_labels` **Document ID format:** `label_XXXXX` where XXXXX is zero-padded label number (e.g., `label_00001`)

AUTHENTICATION SYSTEM

Password encryption (obfuscated, built-in)

The admin password must be stored **encrypted inside the HTML**. Use the following scheme:

Stored hash (SHA-256 of `spiceveg` + site-specific salt):

- Username: `Srikanth`
- Password: `spiceveg`

```javascript
// Obfuscated auth — do NOT name this function "checkPassword"
// Use an innocuous name like `_vx` or `_syncPrefs`
async function _syncPrefs(u, p) {
  // Salt is split across multiple variables to obscure
  const _s1 = "spice";
  const _s2 = "veg_";
  const _s3 = "agri_";
  const _s4 = "2026";
  const salt = _s1 + _s2 + _s3 + _s4;

  const enc = new TextEncoder();
  const hashBuf = await crypto.subtle.digest('SHA-256', enc.encode(p + salt));
  const hashArr = Array.from(new Uint8Array(hashBuf));
  const hashHex = hashArr.map(b => b.toString(16).padStart(2, '0')).join('');

  // Stored hash of "spiceveg" + salt — computed once, hardcoded
  const _h = "e3b2a..." // << REPLACE WITH ACTUAL COMPUTED HASH (see below)

  return u.toLowerCase() === "srikanth" && hashHex === _h;
}
```

IMPORTANT: The agent must compute the actual SHA-256 hash of `"spiceveg" + "spiceveg_agri_2026"` and hardcode it as `_h`. The function name `_syncPrefs` disguises its purpose. The salt variables are split so grep/search won't find a single "password" string. Even if the HTML source is viewed, the hash cannot be reversed.

Session handling:

```javascript
// Store session flag in sessionStorage only (cleared on tab close)
sessionStorage.setItem('_sv_auth', '1');
// Check: if (sessionStorage.getItem('_sv_auth') !== '1') → show login
```

APP ROUTING

The app detects its mode from the URL:

```javascript
function detectMode() {
  const params = new URLSearchParams(window.location.search);
  const labelId = params.get('id');  // e.g. ?id=00001

  if (labelId) {
    // CUSTOMER MODE — QR was scanned, show label details
    loadCustomerView(labelId);
  } else {
    // ADMIN MODE — show login or dashboard
    loadAdminGate();
  }
}
```

QR Code URL format: `https://krishnakoushik9.github.io/Spice-Veg-Agri-Customer/?id=00001`

VISUAL DESIGN SYSTEM

Palette (monochrome + green, flat, no 3D)

```css
:root {
  --bg: #FFFFFF;
  --surface: #F7F9F4;
  --surface2: #EEF3E8;
  --border: #D4DCC8;
  --border-strong: #A8BF94;
  --green-primary: #3B6D11;
  --green-mid: #639922;
  --green-light: #C0DD97;
  --green-pale: #EAF3DE;
  --text-primary: #1A2410;
  --text-secondary: #4A5C3A;
  --text-muted: #7A8F6A;
  --mono: #2C2C2A;
  --danger: #8B2020;
  --danger-bg: #FAEAEA;
}
```

Typography

```css
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600&fa

body { font-family: 'DM Sans', sans-serif; font-size: 15px; line-height: 1.6; back
.brand { font-family: 'Literata', serif; font-weight: 600; }
```

Global rules

- No box shadows. No gradients. No border-radius > 12px.
- All interactive elements have a `0.5px solid var(--border)` border.
- Buttons: flat fill `var(--green-primary)`, white text, `border-radius: 8px`, `padding: 11px 20px`.
- Input fields: `background: var(--surface)`, `border: 1px solid var(--border)`, `border-radius: 6px`, `padding: 9px 12px`.
- Focus ring: `outline: 2px solid var(--green-mid); outline-offset: 2px;`
- Page max-width: `480px`, centered, `padding: 0 16px`.

LOGO RENDERING

Render the SpiceVeg logo as **pure SVG inline** (no image file needed, fast loading):

```html
<!-- Compact SVG logo: green stamp circle, "SPICEVEG" text, "VEGETABLE SEEDS" subt
<svg width="64" height="64" viewBox="0 0 64 64" xmlns="http://www.w3.org/2000/svg"
  <!-- Outer serrated ring -->
  <circle cx="32" cy="32" r="30" fill="#3B6D11" stroke="none"/>
  <!-- Inner yellow circle -->
  <circle cx="32" cy="32" r="22" fill="#F5C518"/>
  <!-- Simple chili icon -->
  <path d="M28 20 Q32 28 30 36" stroke="#C0392B" stroke-width="2.5" fill="none" st
  <path d="M34 22 Q36 30 33 38" stroke="#27AE60" stroke-width="2" fill="none" stro
</svg>
```

For full fidelity: place the actual `logo.png` (from image 3) in the repo root and reference it:

```html
<img src="logo.png" width="64" height="64" alt="SpiceVeg" style="border-radius:50%
```

PAGE STRUCTURE & FLOWS

— FLOW 1: ADMIN LOGIN PAGE —

Shown when: URL has no `?id=` param and user is not authenticated.

Layout:

```
┌──────────────────────────┐
│  [LOGO 64px centered]    │
│  SpiceVeg                │ ← Literata font, green
│  VEGETABLE SEEDS         │ ← small caps, muted
│                          │
│  Admin Login             │ ← h2
│                          │
│  Username  [___________] │
│  Password  [___________] │
│            [  Sign In  ] │
│                          │
│  [error msg in red, hidden] │
└──────────────────────────┘
```

Behavior:

- On submit: call `_syncPrefs(username, password)` async
- If true: `sessionStorage.setItem('_sv_auth', '1')` then `showAdmin()`
- If false: show "Invalid credentials" in `--danger` color
- Password field has show/hide toggle (eye icon as SVG)
- Button shows spinner while async hash runs

HTML snippet:

```html
<div id="login-page" class="page">
  <div class="logo-block">
    <img src="logo.png" width="64" height="64" alt="SpiceVeg Logo">
    <span class="brand" style="font-size:22px;color:var(--green-primary);">SpiceV
    <span style="font-size:11px;letter-spacing:0.1em;color:var(--text-muted);text-
  </div>
  <h2 style="font-size:18px;font-weight:500;margin:28px 0 20px;">Admin Login</h2>
  <div class="field">
    <label>Username</label>
    <input type="text" id="login-user" autocomplete="username" placeholder="Enter
  </div>
  <div class="field">
    <label>Password</label>
    <div style="position:relative;">
      <input type="password" id="login-pass" autocomplete="current-password" place
      <button type="button" id="pw-toggle" style="position:absolute;right:10px;top
        <!-- Eye SVG icon -->
      </button>
    </div>
  </div>
  <div id="login-err" style="display:none;color:var(--danger);font-size:13px;marg
    Invalid username or password.
  </div>
  <button id="login-btn" onclick="doLogin()" style="width:100%;background:var(--gr
    Sign In
  </button>
</div>
```

— FLOW 2: ADMIN DASHBOARD —

Shown after: Successful login.

Tab layout (two tabs at top):

- **New Label** — form to create entry
- **All Labels** — list of past entries from Firestore

TAB A: NEW LABEL FORM

```
┌──────────────────────────────────┐
│  [← Logout]    SpiceVeg Admin    │
│  [New Label] [All Labels]        │
├──────────────────────────────────┤
│  Label Number  [auto: 00001]     │ ← read-only, auto-incremented
│  Crop          [__________]      │ ← e.g. Chilli
│  Variety       [__________]      │ ← e.g. SPICE-55
│  Lot Number    [__________]      │ ← e.g. SV22162
│  Date of Testing [date picker]   │
│  Date of Packaging [date]        │
│  Valid Upto    [date picker]      |
│  Net Weight    [__________]      │ ← e.g. 10g
│  MRP           [__________]      │ ← e.g. 815
│  Germination%  [__________]      │ ← e.g. 92
│  Purity%       [__________]      │ ← e.g. 98
│  Moisture%     [__________]      │ ← e.g. 8
│                                  |
│  [    Generate & Save Label    ] |
│                                  |
│  ── QR Preview ──                |
│  [QR code renders here]          |
│  URL: https://.../?id=00001      |
│  [ ⬇ Download QR PNG ]           |
│  [ 🖨 Print Label ]              |
└──────────────────────────────────┘
```

Label number auto-increment logic:

```javascript
async function getNextLabelNumber() {
  const all = await fsList('seed_labels');
  if (!all.length) return '00001';
  const nums = all.map(d => parseInt(d.labelNo || '0')).filter(n => !isNaN(n));
  const max = Math.max(...nums, 0);
  return String(max + 1).padStart(5, '0');
}
```

On "Generate & Save Label":

1. Validate all fields (show inline errors if empty)
2. Build data object with all fields + `createdAt: new Date().toISOString()`
3. Call `fsSet('seed_labels', 'label_' + labelNo, data)`
4. Show success toast: "Label 00001 saved ✓"
5. Generate QR code pointing to `https://krishnakoushik9.github.io/Spice-Veg-Agri-Customer/?id=00001`
6. Show QR in preview div + Download + Print buttons

QR Code generation:

```javascript
function generateQR(labelNo) {
  document.getElementById('qr-box').innerHTML = '';
  const url = `https://krishnakoushik9.github.io/Spice-Veg-Agri-Customer/?id=${lab
  new QRCode(document.getElementById('qr-box'), {
    text: url,
    width: 200,
    height: 200,
    colorDark: "#1A2410",
    colorLight: "#FFFFFF",
    correctLevel: QRCode.CorrectLevel.H
  });
  document.getElementById('qr-url').textContent = url;
  document.getElementById('qr-section').style.display = 'block';
}
```

Download QR as PNG:

```javascript
function downloadQR(labelNo) {
  const canvas = document.querySelector('#qr-box canvas');
  if (!canvas) return;
  // Add logo watermark to canvas before download
  const ctx = canvas.getContext('2d');
  const link = document.createElement('a');
  link.download = `SpiceVeg_Label_${labelNo}.png`;
  link.href = canvas.toDataURL('image/png');
  link.click();
}
```

Print label function:

javascript

```javascript
function printLabel(data) {
  const canvas = document.querySelector('#qr-box canvas');
  const qrDataUrl = canvas ? canvas.toDataURL() : '';
  const w = window.open('', '_blank');
  w.document.write(`
    <html><head><title>SpiceVeg Label ${data.labelNo}</title>
    <style>
      body { font-family: Arial, sans-serif; font-size: 10px; margin: 0; padding:
      .brand { font-size: 14px; font-weight: bold; color: #3B6D11; }
      .row { display: flex; justify-content: space-between; margin: 1mm 0; }
      img.qr { width: 25mm; height: 25mm; }
    </style></head><body>
    <div class="brand">SpiceVeg™</div>
    <small>VEGETABLE SEEDS</small>
    <hr>
    <div class="row"><b>TRUTHFUL LABEL</b></div>
    <div>Label No: ${data.labelNo} | Crop: ${data.crop}</div>
    <div>Variety: ${data.variety}</div>
    <div>Lot No: ${data.lotNo} | DOT: ${data.dot}</div>
    <div>DOP: ${data.dop} | Valid Upto: ${data.validUpto}</div>
    <div>Net Wt: ${data.netWeight} | MRP: ₹${data.mrp}/- (Incl. of all taxes)</div
    <hr>
    <img class="qr" src="${qrDataUrl}" alt="QR Code">
    </body></html>
  `);
  w.document.close();
  w.print();
}
```

TAB B: ALL LABELS LIST

```
┌──────────────────────────────────┐
│  All Labels          [🔄 Refresh]  │
├──────────────────────────────────┤
│  #00001 — Chilli / SPICE-55       │
│  Lot: SV22162  |  Valid: 20-01-2027│
│  [View QR]  [Edit]  [Open ↗]      │
├──────────────────────────────────┤
│  #00002 — Tomato / HYB-RED        │
│  Lot: SV22163  |  Valid: 15-06-2027│
│  [View QR]  [Edit]  [Open ↗]      │
└──────────────────────────────────┘
```

"Edit" flow: Pre-fills the New Label form with that record's data. Updates existing Firestore doc on save instead of creating new. Shows "Update Label" button instead of "Generate & Save".

"View QR" flow: Opens a modal overlay with the regenerated QR for that label + download button.

Modal (no `position:fixed` — use faux viewport wrapper):

```html
<div id="modal-wrap" style="display:none;min-height:100vh;background:rgba(0,0,0,0
  <div id="modal-inner" style="background:var(--bg);border-radius:12px;padding:24p
    <div id="modal-qr"></div>
    <p id="modal-url" style="font-size:11px;color:var(--text-muted);word-break:bre
    <button onclick="downloadModalQR()" style="...">Download PNG</button>
    <button onclick="closeModal()" style="...margin-top:8px;">Close</button>
  </div>
</div>
```

— FLOW 3: CUSTOMER PAGE (QR SCAN) —

Triggered when: URL contains `?id=XXXXX`

Behavior on load:

1. Parse `id` from URL params
2. Fetch from Firestore: `fsGet('seed_labels', 'label_' + id)`
3. Show loading spinner while fetching
4. If found: render label details (read-only)
5. If not found: show "Label not found" error card

IMPORTANT — Back button protection:

```javascript
// Push a dummy history entry so Back doesn't go to admin
history.pushState(null, '', window.location.href);
window.addEventListener('popstate', () => {
  history.pushState(null, '', window.location.href);
});
```

Customer page layout:

```
┌──────────────────────────────┐
│  [LOGO]  SpiceVeg™           │
│          VEGETABLE SEEDS     │
│  ✓ QR Verified               │  ← green badge
├──────────────────────────────┤
│  TRUTHFUL LABEL              │  ← bold, uppercase, green
│                              │
│  Label No.      00001        │
│  Crop           Chilli       │
│  Variety        SPICE-55     │
│  Lot No.        SV22162      │
│  ────────────────────────────│
│  Date of Testing  24-04-2026 │
│  Date of Pkg      24-04-2026 │
│  Valid Upto     20-01-2027   │
│  ────────────────────────────│
│  Net Weight     10g          │
│  MRP            ₹815/-       │
│  (Incl. of all taxes)        │
├──────────────────────────────┤
│  Lot Quality                 │
│  Germination  ▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 92%  │
│  Purity       ▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 98%  │
│  Moisture     ▓▓▓▓▓▓▓▓▓▓▓▓▓▓  8%  │
├──────────────────────────────┤
│  [View Cultivation Practices]│ ← green pill button, full width
└──────────────────────────────┘
```

Progress bar component:

html

```html
<div class="quality-bar">
  <span class="qb-label">Germination</span>
  <div class="qb-track">
    <div class="qb-fill" style="width: 92%;"></div>
  </div>
  <span class="qb-pct">92%</span>
</div>
```

css

```css
.qb-track { flex:1; height:6px; background:var(--surface2); border-radius:3px; mar
.qb-fill { height:100%; background:var(--green-primary); border-radius:3px; trans
```

"View Cultivation Practices" button:

```javascript
function openCultivation(crop) {
  // Opens technique_[crop].png in a lightbox overlay
  // Filename: technique_chilli.png, technique_tomato.png etc.
  const fname = 'technique_' + crop.toLowerCase().replace(/\s+/g, '_') + '.png';
  showImageLightbox(fname);
}

function showImageLightbox(src) {
  const lb = document.getElementById('lightbox');
  document.getElementById('lb-img').src = src;
  lb.style.display = 'flex';
  history.pushState(null, '', window.location.href); // prevent back
}
```

Lightbox HTML:

```html
<div id="lightbox" style="display:none;position:absolute;top:0;left:0;width:100%;r
  <div style="position:relative;max-width:480px;width:100%;">
    <button onclick="closeLightbox()" style="position:absolute;top:-40px;right:0;b
    <img id="lb-img" src="" alt="Cultivation Practices" style="width:100%;border-r
  </div>
</div>
```

Image file naming convention (place these in repo root):

- `technique_chilli.png`
- `technique_tomato.png`
- `technique_watermelon.png`
- *(etc. for each crop)*

DATA MODEL (Firestore document)

Each document in collection `seed_labels`, doc ID `label_XXXXX`:

```javascript
{
  labelNo: "00001",           // string, zero-padded 5 digits
  crop: "Chilli",
  variety: "SPICE-55",
  lotNo: "SV22162",
  dot: "24-04-2026",          // Date of Testing
  dop: "24-04-2026",          // Date of Packaging
  validUpto: "20-01-2027",
  netWeight: "10g",
  mrp: "815",
  germinationRate: "92",      // percentage string
  purity: "98",
  moistureLevel: "8",
  createdAt: "2026-04-23T..."  // ISO timestamp
}
```

COMPLETE FILE STRUCTURE

```
Spice-Veg-Agri-Customer/
├── index.html          ← entire app
├── logo.png            ← SpiceVeg logo (from image 3)
├── technique_chilli.png
├── technique_tomato.png
├── technique_watermelon.png
└── README.md
```

HTML FILE SECTIONS ORDER

Build `index.html` in this exact order:

```
1. <head>
   - meta charset, viewport (mobile-first)
   - title: SpiceVeg — Seed Verification
   - Google Fonts async link
   - <style> block with all CSS variables + global styles

2. <body>
   - #loading-screen (spinner, shows on first load)
   - #login-page (admin login form, hidden by default)
   - #admin-page (dashboard with tabs, hidden by default)
     - #tab-new (new label form)
     - #tab-list (all labels table)
     - #qr-section (QR preview, download, print)
     - #modal-wrap (QR modal for list view)
   - #customer-page (QR-scanned view, hidden by default)
     - #lightbox (cultivation image overlay)

3. <script src="qrcode.min.js CDN">
4. <script> all application logic:
   - Constants (FB config, FS_BASE)
   - Firestore helpers (fsSet, fsGet, fsList)
   - Auth (_syncPrefs, doLogin, doLogout)
   - Router (detectMode, on DOMContentLoaded)
   - Admin logic (loadAdmin, loadLabelList, savLabel, generateQR, downloadQR,
printLabel)
   - Customer logic (loadCustomerView, renderLabel, openCultivation,
showImageLightbox)
   - UI helpers (showPage, showToast, showSpinner)
```

UX MICRO-INTERACTIONS

Toast notification

```javascript
function showToast(msg, type = 'success') {
  const t = document.createElement('div');
  t.className = 'toast';
  t.textContent = msg;
  t.style.cssText = `
    position:fixed;bottom:24px;left:50%;transform:translateX(-50%);
    background:${type==='success'?'var(--green-primary)':'var(--danger)'};
    color:#fff;padding:10px 20px;border-radius:8px;font-size:14px;
    z-index:999;animation:fadeup 0.3s ease;
  `;
  document.body.appendChild(t);
  setTimeout(() => t.remove(), 2800);
}
```

Loading spinner (inline SVG, no gif)

```html
<div id="loading-screen" style="display:flex;align-items:center;justify-content:ce
  <svg width="40" height="40" viewBox="0 0 40 40">
    <circle cx="20" cy="20" r="16" fill="none" stroke="#C0DD97" stroke-width="3"/>
    <circle cx="20" cy="20" r="16" fill="none" stroke="#3B6D11" stroke-width="3"
      stroke-dasharray="50 50" stroke-linecap="round">
      <animateTransform attributeName="transform" type="rotate" from="0 20 20" to=
    </circle>
  </svg>
  <span style="color:var(--text-muted);font-size:13px;">Loading...</span>
</div>
```

Field validation inline errors

```javascript
function validateField(id, msg) {
  const el = document.getElementById(id);
  let err = document.getElementById(id + '-err');
  if (!el.value.trim()) {
    if (!err) {
      err = document.createElement('span');
      err.id = id + '-err';
      err.style.cssText = 'color:var(--danger);font-size:12px;display:block;margin
      el.parentNode.appendChild(err);
    }
    err.textContent = msg;
    el.style.borderColor = 'var(--danger)';
    return false;
  }
  if (err) err.remove();
  el.style.borderColor = '';
  return true;
}
```

Tab switching

```javascript
function switchTab(tab) {
  ['new', 'list'].forEach(t => {
    document.getElementById('tab-' + t).style.display = t === tab ? 'block' : 'non
    document.getElementById('tabn-' + t).classList.toggle('active', t === tab);
  });
  if (tab === 'list') loadLabelList();
}
```

SECURITY NOTES

- Admin login page is entirely hidden when `?id=` is in URL — the DOM elements are removed/not rendered
- `sessionStorage` (not `localStorage`) means session ends when tab closes
- No admin routes exposed in customer mode — separate `showPage()` function ensures only one page div is visible at a time
- Back button hijacked in customer mode via `history.pushState` loop
- Firebase rules are already set to `allow read, write: if true` — this is fine for this use case since:
  - Write access only happens after admin auth (enforced client-side)
  - Label data is non-sensitive (seed product info only)
  - No PII stored

STEP-BY-STEP BUILD CHECKLIST FOR AGENT

```
[ ] 1. Create index.html skeleton with all page divs
[ ] 2. Add CSS variables, global styles, mobile viewport
[ ] 3. Import QRCode.js via CDN script tag
[ ] 4. Implement Firestore REST helpers (fsSet, fsGet, fsList)
[ ] 5. Implement _syncPrefs auth with SHA-256 (compute actual hash first!)
[ ] 6. Implement detectMode() router on DOMContentLoaded
[ ] 7. Build login page HTML + doLogin() logic
[ ] 8. Build admin new label form HTML with all fields
[ ] 9. Implement getNextLabelNumber() + saveLabel() + fsSet call
[ ] 10. Implement generateQR() using QRCode.js
[ ] 11. Implement downloadQR() canvas → PNG
[ ] 12. Implement printLabel() print window
[ ] 13. Build all labels list tab + loadLabelList() + fsList call
[ ] 14. Build QR modal for list view
[ ] 15. Build customer page HTML (read-only label display)
[ ] 16. Implement loadCustomerView() + fsGet + renderLabel()
[ ] 17. Implement back button protection (history.pushState loop)
[ ] 18. Implement openCultivation() + showImageLightbox()
[ ] 19. Add loading spinner, toast notifications, inline validation
[ ] 20. Test: admin login → create label → QR generates → open in new tab with
?id= → customer view shows correct data
[ ] 21. Test: customer page back button stays on customer page
[ ] 22. Verify: no admin elements visible in customer mode
[ ] 23. Add README.md with deploy instructions
[ ] 24. Commit and push to GitHub, enable GitHub Pages from root/main
```

DEFAULT SAMPLE DATA (pre-populate for testing)

When building, have the admin form pre-fill with:

```
Crop: Chilli
Variety: SPICE-55
Lot No: SV22162
Date of Testing: 24-04-2026
Date of Packaging: 24-04-2026
Valid Upto: 20-01-2027
Net Weight: 10g
MRP: 815
Germination Rate: 92
Purity: 98
Moisture Level: 8
```

(Clear before final production, but useful for testing)

IMPORTANT REMINDERS

1. **Single file** — everything in `index.html`. No external CSS file. No external JS file.
2. **No SDK** — all Firebase calls via `fetch()` to REST API only.
3. **QR codes must be scannable** — use `correctLevel: QRCode.CorrectLevel.H` (highest error correction).
4. **Font loading must be non-blocking** — use `<link rel="preconnect">` + `display=swap`.
5. **Admin page must be completely invisible from customer view** — not just `display:none`, but never rendered.
6. **Compute the SHA-256 hash before hardcoding** — run in browser console:

```javascript
const enc = new TextEncoder();
const buf = await crypto.subtle.digest('SHA-256', enc.encode('spicevegspiceveg_
const hex = Array.from(new Uint8Array(buf)).map(b=>b.toString(16).padStart(2,'0
console.log(hex);
```

Then paste the output as the value of `_h` in `_syncPrefs`.

Prompt prepared for SpiceVeg Agri — Project Pramana build agent, April 2026